

CSE 255 – Lecture 5

Data Mining and Predictive Analytics

Recommender Systems

Why recommendation?

The goal of recommender systems is...

- To help people discover new content

Recommendations for You in Amazon Instant Video [See more](#)



Why recommendation?

- The goal of recommender systems is...
- To help us find the content we were already looking for



Customers Who Watched This Item Also Watched



Why recommendation?

The goal of recommender systems is...

- To discover which things go together



Calvin Klein Men's Relaxed Straight Leg Jean In Cove

★★★★☆ · 20 customer reviews

Price: ~~\$48.16~~ - ~~\$69.99~~ & FREE Returns. [Details](#)

Size:

Select [Sizing info](#) | [Fit: As expected \(55%\)](#)

Color: Cove

- 98% Cotton/2% Elastane
- Imported
- Button closure
- Machine Wash
- Relaxed straight-leg jean in light-tone denim featuring whiskering and five-pocket styling
- Zip fly with button
- 10.25-inch front rise, 19-inch knee, 17.5-inch leg opening

Frequently Bought Together



Calvin Klein Jeans
\$57.94 - \$69.50



Calvin Klein Jeans
\$49.92



Calvin Klein Jeans
\$50.67 - \$69.99



Levi's
\$23.99 - \$68.00

Customers Who Bought This Item Also Bought

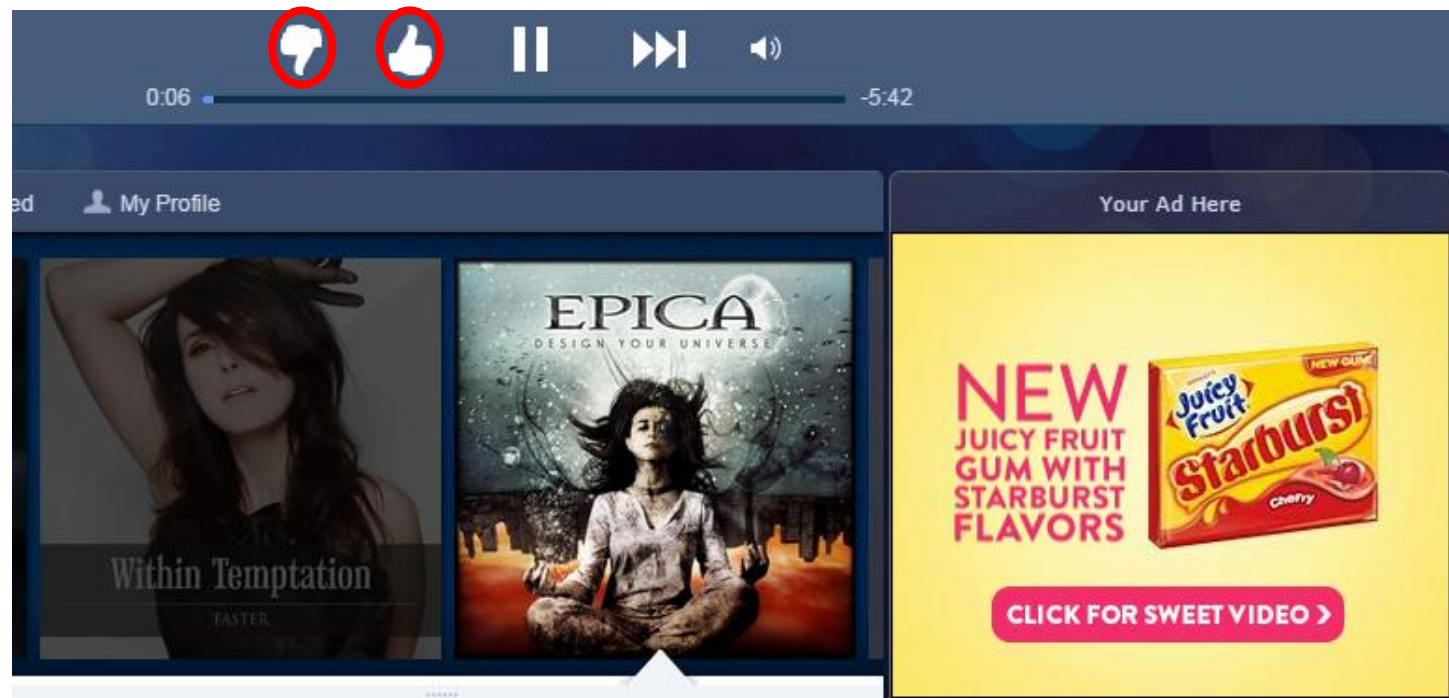


Page 3

Why recommendation?

The goal of recommender systems is...

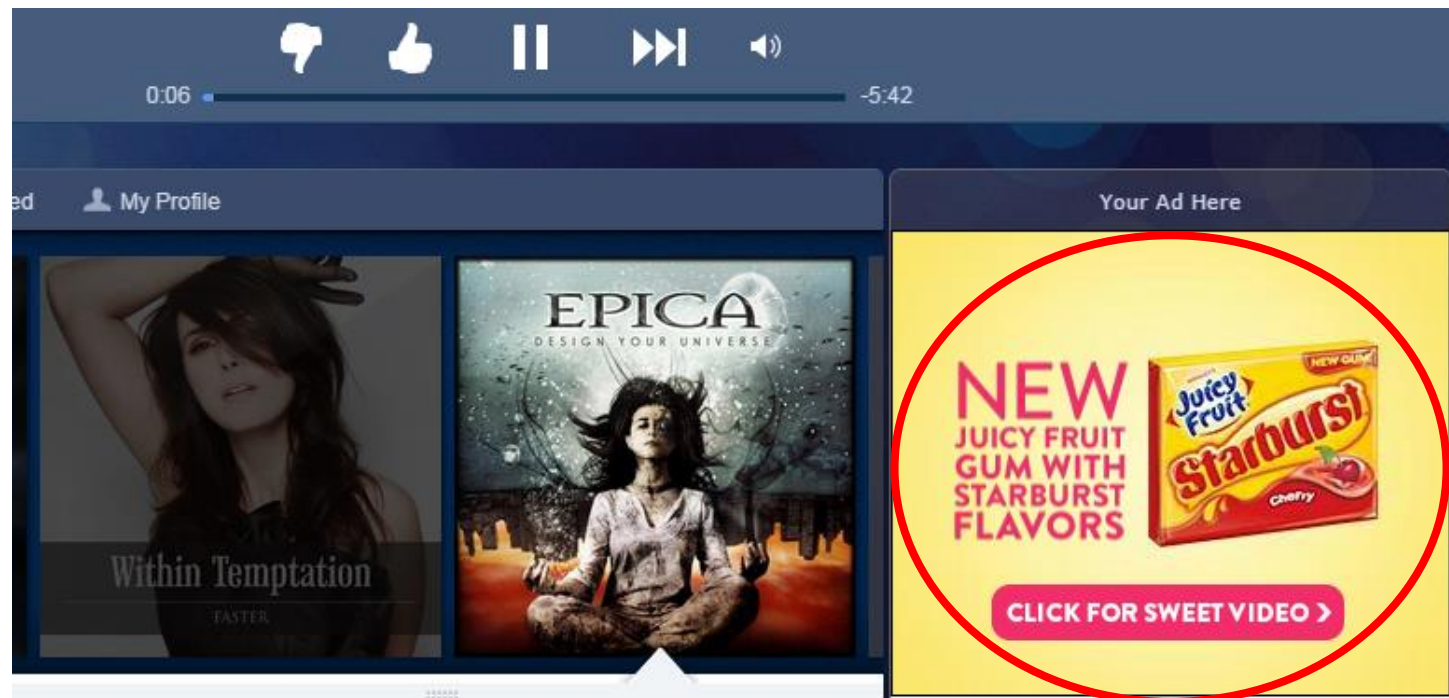
- To personalize user experiences in response to user feedback



Why recommendation?

The goal of recommender systems is...

- To recommend incredible products that are relevant to our interests

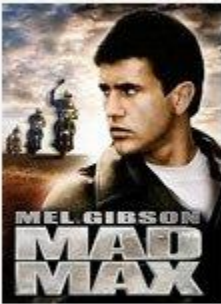


Why recommendation?

The goal of recommender systems is...

- To identify things that we **like**

Results for 'mad max'



Add

★★★★☆

Mad Max
1979 **R** 93 minutes

In a postapocalyptic future, jaded motorcycle cop Max Rockatansky is ready to retire. But his world is shattered when a malicious gang murders his family as an act of retaliation, forcing a devastated Max to hit the open road seeking vengeance.

Starring: Mel Gibson, Hugh Keays-Byrne
Director: George Miller
Genre: Sci-Fi & Fantasy
Format: DVD

★★★★☆ **3.6** Our best guess for Jeremy

Why recommendation?

The goal of recommender systems is...

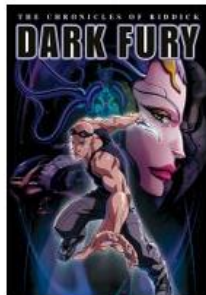
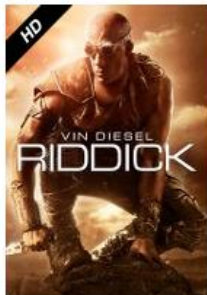
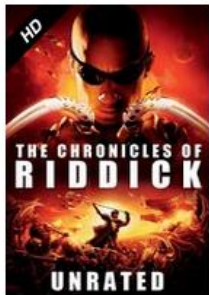
- To help people discover new content
- To help us find the content we were
- To discover preferences, opinions, together
- To provide recommendations in response to user feedback
- To identify things that we **like**

To **model** people's
preferences, opinions,
and behavior

Recommending things to people

Suppose we want to build a movie recommender

e.g. which of these films will I rate highest?



Recommending things to people

We already have
a few tools in our
“supervised
learning” toolbox
that may help us

Pitch Black - Unrated Director's Cut [R] [CC]

★★★★★ 777 **IMDb** 7.1/10

Watch Trailer

When their ship crash-lands on a remote planet, the marooned passengers soon learn that escaped convict Riddick (Vin Diesel) isn't the only thing they have to fear. Deadly creatures lurk in the shadows, waiting to attack in the dark, and the planet is rapidly plunging into the

See More

Starring: Vin Diesel, Radha Mitchell
Runtime: 1 hour, 53 minutes
Available to watch on supported devices.

Product Details

Genres	Science Fiction, Action, Horror
Director	David Twohy
Starring	Vin Diesel, Radha Mitchell
Supporting actors	Cole Hauser, Keith David, Lewis Fitz-Gerald, Claudia Black, Rhiana Granger, Angela Moore, Peter Chiang, Ken Twohy
Studio	NBC Universal
MPAA Rating	R (Restricted)
Options and subtitles	English Details
Rental rights	24 hour viewing period. Details
Purchase rights	Stream instantly and download to 2 locations Details
Format	Amazon Instant Video (streaming online video and digital download)

A. Phillips
Reviewer ranking: #17,230,554
90% helpful
votes received on reviews (151 of 167)

ABOUT ME
Enjoy the reviews...

ACTIVITIES
[Reviews \(16\)](#)
[Public Wish List \(2\)](#)
[Listmania Lists \(2\)](#)
[Tagged Items \(1\)](#)

$f(\text{user features, movie features}) \xrightarrow{?} \text{star rating}$

Recommending things to people

$f(\text{user features, movie features}) \xrightarrow{?} \text{star rating}$

Movie features: genre, actors, rating, length, etc.

Product Details

Genres	Science Fiction , Action , Horror
Director	David Twohy
Starring	Vin Diesel , Radha Mitchell
Supporting actors	Cole Hauser , Keith David , Lewis Fitz-Gerald , Claudia Black , Rhiana Granger , Angela Moore , Peter Chiang , Ken Twohy
Studio	NBC Universal
MPAA rating	R (Restricted)
Captions and subtitles	English Details ▾
Rental rights	24 hour viewing period. Details ▾
Purchase rights	Stream instantly and download to 2 locations Details ▾
Format	Amazon Instant Video (streaming online video and digital download)

User features: age, gender, location, etc.

A. Phillips

Reviewer ranking: #17,230,554

90% helpful

votes received on reviews
(151 of 167)

ABOUT ME

Enjoy the reviews...

ACTIVITIES

[Reviews](#) (16)

[Public Wish List](#) (2)

[Listmania Lists](#) (2)

[Tagged Items](#) (1)

Recommending things to people

$f(\text{user features, movie features}) \xrightarrow{?} \text{star rating}$

With the models we've seen so far, we can build predictors that account for...

- Do women give higher ratings than men?
- Do Americans give higher ratings than Australians?
- Do people give higher ratings to action movies?
- Are ratings higher in the summer or winter?
- Do people give high ratings to movies with Vin Diesel?

So what **can't** we do yet?

Recommending things to people

$f(\text{user features}, \text{movie features}) \xrightarrow{?} \text{star rating}$

Consider the following linear predictor
(e.g. from week 1):

$$\begin{aligned} f(\text{user features}, \text{movie features}) &= \langle \phi(\text{user features}); \phi(\text{movie features}), \theta \rangle \\ &= \langle \phi(\text{user features}), \theta_{\text{user}} \rangle + \langle \phi(\text{movie features}), \theta_{\text{movie}} \rangle \end{aligned}$$


Recommending things to people

But this is essentially just two separate predictors!

$$\begin{aligned} f(\text{user features}, \text{movie features}) = \\ = \underbrace{\langle \phi(\text{user features}), \theta_{\text{user}} \rangle}_{\text{user predictor}} + \underbrace{\langle \phi(\text{movie features}), \theta_{\text{movie}} \rangle}_{\text{movie predictor}} \end{aligned}$$

That is, we're treating user and movie features as though they're **independent**

Recommending things to people

But these predictors should (obviously?)
not be independent

$$f(\text{user features, movie features}) = f(\text{user}) + f(\text{movie})$$

do I tend to give high ratings?



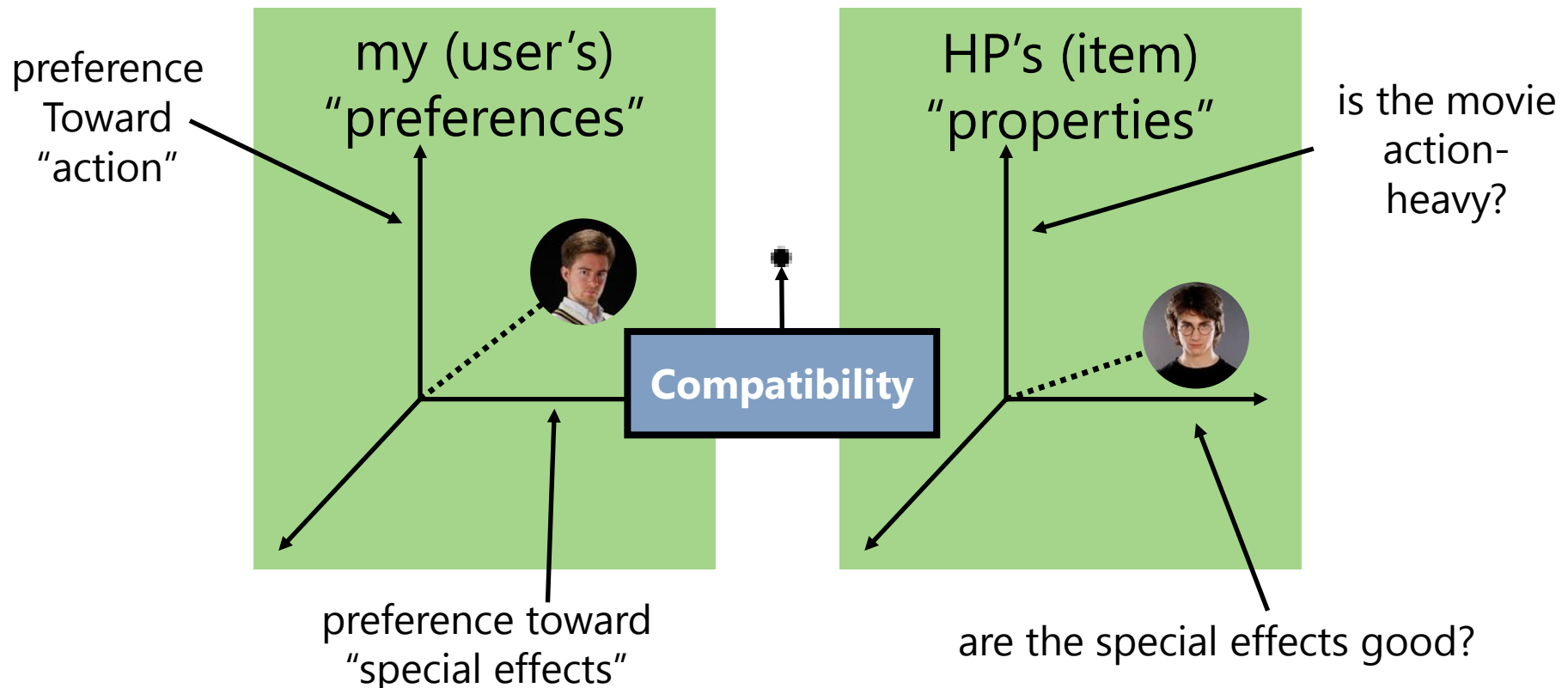
does the population tend to give high ratings to this genre of movie?



But what about a feature like "do **I** give
high ratings to **this genre** of movie"?

Recommending things to people

Recommender Systems go beyond the methods we've seen so far by trying to model the **relationships** between people and the items they're evaluating



Today

Recommender Systems

1. Collaborative filtering

(performs recommendation in terms of user/user and item/item similarity)

2. Latent-factor models

(performs recommendation by projecting users and items into some low-dimensional space)

Defining similarity between users & items

Q: How can we measure the **similarity** between two **users**?

A: In terms of the **items** they purchased!

Q: How can we measure the similarity between two **items**?

A: In terms of the users who purchased them!

Defining similarity between users & items

e.g.:
Amazon



Calvin Klein Men's Relaxed Straight Leg Jean In Cove

★★★★★ 20 customer reviews

Price: \$48.16 - \$69.99 & FREE Returns. Details

Size:

Select Sizing info Fit: As expected (55%)

Color: Cove

- 98% Cotton/2% Elastane
- Imported
- Button closure
- Machine Wash
- Relaxed straight-leg jean in light-tone denim featuring whiskering and five-pocket styling
- Zip fly with button
- 10.25-inch front rise, 19-inch knee, 17.5-inch leg opening

Frequently Bought Together



Calvin Klein Jeans
\$57.94 - \$69.50



Calvin Klein Jeans
\$49.92



Calvin Klein Jeans
\$50.67 - \$69.99



Levi's
\$23.99 - \$68.00

Customers Who Viewed This Item Also Viewed



Customers Who Bought This Item Also Bought



Definitions

Definitions

I_u = set of items purchased by user u

U_i = set of users who purchased item i

Definitions

Or equivalently...

$$R = \begin{pmatrix} 1 & 0 & \dots & 1 \\ 0 & 0 & & 1 \\ \vdots & & \ddots & \vdots \\ 1 & 0 & \dots & 1 \end{pmatrix}$$

items

users

R_u = binary representation items purchased by u

$R_{.,i}$ = binary representation of users who purchased i

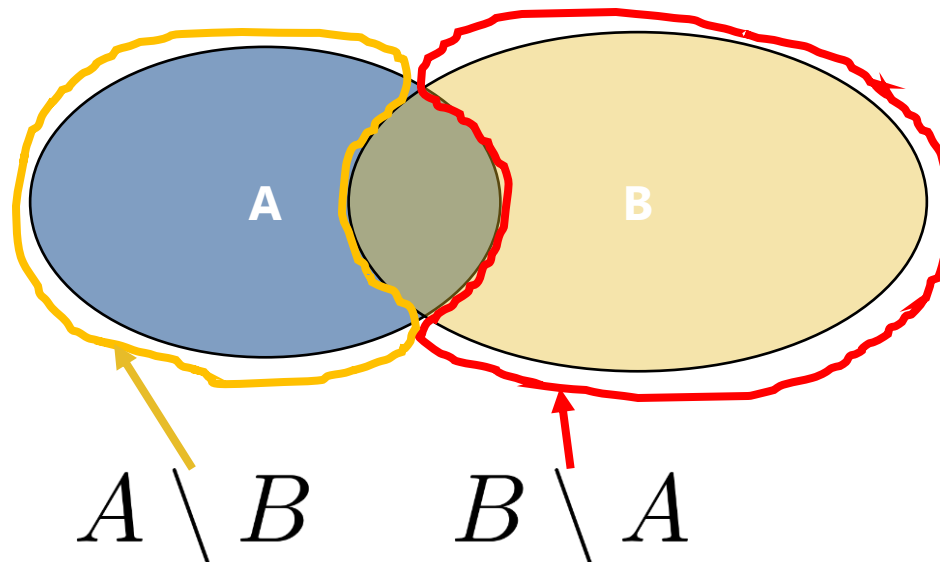
$$I_u = \{i | R_{u,i} = 1\} \quad U_i = \{u | R_{u,i} = 1\}$$

0. Euclidean distance

Euclidean distance:

e.g. between two items i, j (similarly defined between two users)

$$|U_i \setminus U_j| + |U_j \setminus U_i| = \|R_i - R_j\|$$



0. Euclidean distance

Euclidean distance:

e.g.: $U_1 = \{1, 4, 8, 9, 11, 23, 25, 34\}$
 $U_2 = \{1, 4, 6, 8, 9, 11, 23, 25, 34, 35, 38\}$
 $U_3 = \{4\}$
 $U_4 = \{5\}$

$$|U_1 \setminus U_2| + |U_2 \setminus U_1| = 3$$

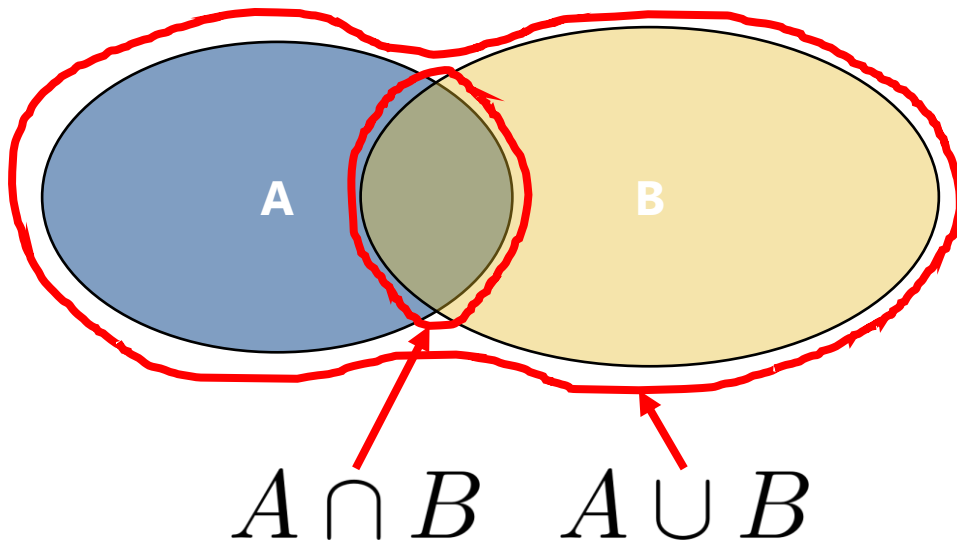
$$|U_3 \setminus U_4| + |U_4 \setminus U_3| = 2$$

Problem: favors small sets, even if they have few elements in common

1. Jaccard similarity

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

$$\text{Jaccard}(U_i, U_j) = \frac{|U_i \cap U_j|}{|U_i \cup U_j|}$$



→ Maximum of 1 if the two users purchased **exactly the same** set of items

(or if two items were purchased by the same set of users)

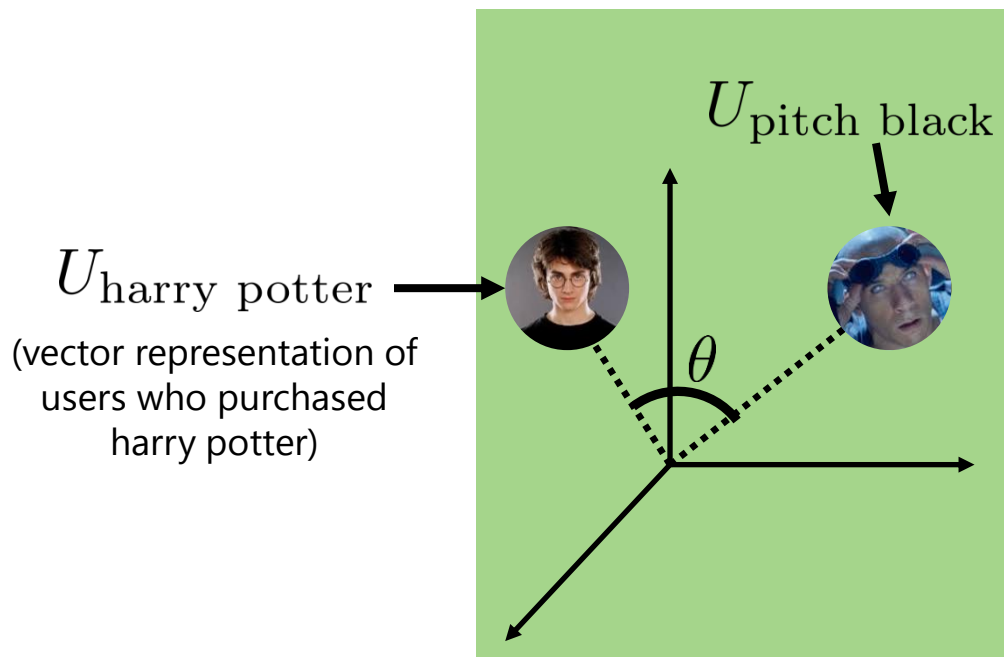
→ Minimum of 0 if the two users purchased **completely disjoint** sets of items

(or if the two items were purchased by completely disjoint sets of users)

2. Cosine similarity

$$\text{Cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

$$\theta = \cos^{-1} \left(\frac{A \cdot B}{\|A\| \|B\|} \right)$$



$$\cos(\theta) = 1$$

(theta = 0) $\rightarrow$ A and B point in exactly the same direction

$$\cos(\theta) = -1$$

(theta = 180) $\rightarrow$ A and B point in opposite directions (won't actually happen for 0/1 vectors)

$$\cos(\theta) = 0$$

(theta = 90) $\rightarrow$ A and B are orthogonal

2. Cosine similarity

Why cosine?

- Unlike Jaccard, works for arbitrary vectors
- E.g. what if we have **opinions** in addition to purchases?

$$R = \begin{pmatrix} 1 & 0 & \dots & 1 \\ 0 & 0 & & 1 \\ \vdots & & \ddots & \vdots \\ 1 & 0 & \dots & 1 \end{pmatrix} \longrightarrow \begin{pmatrix} -1 & 0 & \dots & 1 \\ 0 & 0 & & -1 \\ \vdots & & \ddots & \vdots \\ 1 & 0 & \dots & -1 \end{pmatrix}$$

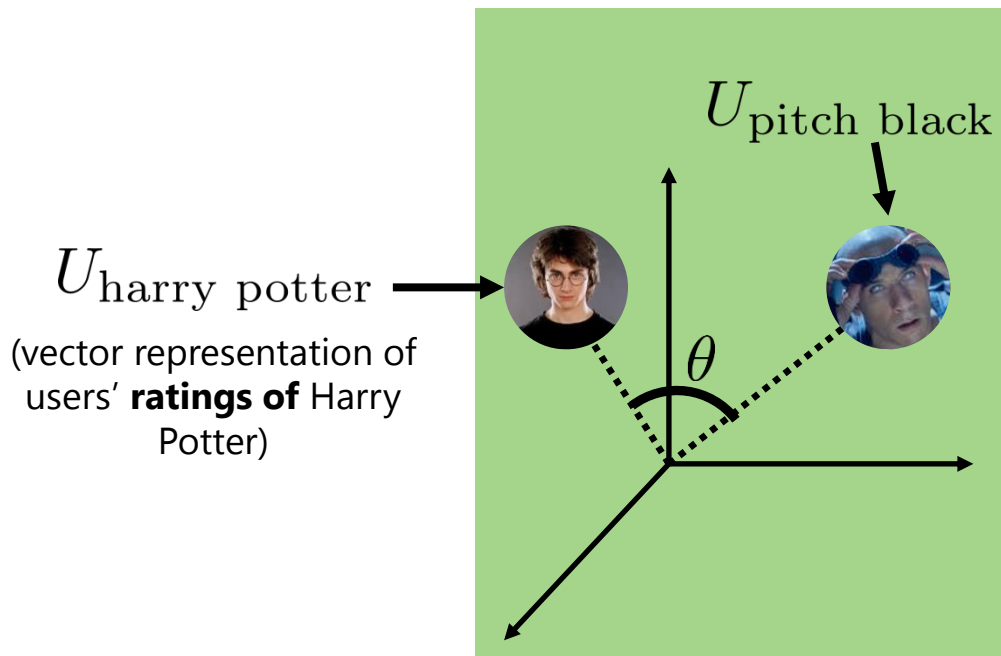
bought and **liked**

didn't buy

bought and **hated**

2. Cosine similarity

E.g. our previous example, now with “thumbs-up/thumbs-down” ratings



$$\cos(\theta) = 1$$

($\theta = 0$) $\rightarrow$ Rated by the
same users, and they all agree

$$\cos(\theta) = -1$$

($\theta = 180$) $\rightarrow$ Rated by the
same users, but they
completely disagree about it

$$\cos(\theta) = 0$$

($\theta = 90$) $\rightarrow$ Rated by
different sets of users

4. Pearson correlation

What if we have numerical ratings
(rather than just thumbs-up/down)?

$$R = \begin{pmatrix} -1 & 0 & \dots & 1 \\ 0 & 0 & & -1 \\ \vdots & & \ddots & \vdots \\ 1 & 0 & \dots & -1 \end{pmatrix} \longrightarrow \begin{pmatrix} 4 & 0 & \dots & 2 \\ 0 & 0 & & 3 \\ \vdots & & \ddots & \vdots \\ 5 & 0 & \dots & 1 \end{pmatrix}$$

bought and **liked**

didn't buy

bought and **hated**

4. Pearson correlation

What if we have numerical ratings
(rather than just thumbs-up/down)?

- We wouldn't want 1-star ratings to be parallel to 5-star ratings
- So we can subtract the average – values are then **negative** for below-average ratings and **positive** for above-average ratings

$$\text{Sim}(u, v) = \frac{\sum_{i \in I_u \cap I_v} (R_{u,i} - \bar{R}_u)(R_{v,i} - \bar{R}_v)}{\sqrt{\sum_{i \in I_u \cap I_v} (R_{u,i} - \bar{R}_u)^2 \sum_{i \in I_u \cap I_v} (R_{v,i} - \bar{R}_v)^2}}$$

items rated by both users average rating by user v

4. Pearson correlation

Compare to the cosine similarity:

Pearson similarity (between users):

$$\text{Sim}(u, v) = \frac{\sum_{i \in I_u \cap I_v} (R_{u,i} - \bar{R}_u)(R_{v,i} - \bar{R}_v)}{\sqrt{\sum_{i \in I_u \cap I_v} (R_{u,i} - \bar{R}_u)^2 \sum_{i \in I_u \cap I_v} (R_{v,i} - \bar{R}_v)^2}}$$

items rated by both users average rating by user v

Cosine similarity (between users):

$$\text{Sim}(u, v) = \frac{\sum_{i \in I_u \cap I_v} R_{u,i} R_{v,i}}{\sqrt{\sum_{i \in I_u \cap I_v} R_{u,i}^2 \sum_{i \in I_u \cap I_v} R_{v,i}^2}}$$

Collaborative filtering in practice

How did Amazon generate their ground-truth data?

Given a product:



Let U_i be the set of users who viewed it

Rank products according to: $\frac{|U_i \cap U_j|}{|U_i \cup U_j|}$ (or cosine/pearson)



Collaborative filtering in practice

Note: (surprisingly) that we built something pretty useful out of **nothing but rating data** – we didn't look at any features of the products whatsoever

Collaborative filtering in practice

But: we still have
a few problems left to address...

1. This is actually kind of slow given a huge enough dataset – if one user purchases one item, this will change the rankings of **every other item that was purchased by at least one user in common**
2. Of no use for **new users** and **new items** (“cold-start” problems)
3. Won’t necessarily encourage diverse results

Questions

CSE 255 – Lecture 5

Data Mining and Predictive Analytics

Latent-factor models

Latent factor models

So far we've looked at approaches that try to define some definition of user/user and item/item **similarity**

Recommendation then consists of

- Finding an item i that a user likes (gives a high rating)
- Recommending items that are similar to it (i.e., items j with a similar rating profile to i)

Latent factor models

What we've seen so far are **unsupervised** approaches and whether the work depends highly on whether we chose a "good" notion of similarity

So, can we perform recommendations via **supervised** learning?

Latent factor models

e.g. if we can model

$f(\text{user features, movie features}) \rightarrow \text{star rating}$

Then recommendation
will consist of identifying

$$\textit{recommendation}(u) = \arg \max_{i \in \text{unseen items}} f(u, i)$$

The Netflix prize

In 2006, Netflix created a dataset of **100,000,000** movie ratings
Data looked like:

(userID, itemID, time, rating)

The goal was to reduce the (R)MSE at predicting ratings:

$$\text{RMSE}(f) = \sqrt{\frac{1}{N} \sum_{u,i,t \in \text{test set}} (f(u,i,t) - r_{u,i,t})^2}$$


model's prediction ground-truth

Whoever first manages to reduce the RMSE by **10%** versus
Netflix's solution wins **\$1,000,000**

The Netflix prize

This led to **a lot** of research on rating prediction by minimizing the Mean-Squared Error

NETFLIX

(it also led to a lawsuit against Netflix, once somebody managed to de-anonymize their data)

We'll look at a few of the main approaches

Rating prediction

Let's start with the
simplest possible model:

$$f(u, i) = \alpha$$


user item

$$\alpha = \frac{1}{N} \sum_{u, i \in \text{training data}} r_{u, i}$$

Here the RMSE is just equal to the
standard deviation of the data

(and we cannot do any better with a 0th order predictor)

Rating prediction

What about the **2nd** simplest model?

$$f(u, i) = \alpha + \beta_u + \beta_i$$

user item

how much does
this user tend to
rate things above
the mean?

does this item tend
to receive higher
ratings than others

e.g.

$$\alpha = 4.2$$



$$\beta_{\text{pitch black}} = -0.1$$

$$\beta_{\text{julian}} = -0.2$$



Rating prediction

The optimization problem becomes:

$$\arg \min_{\alpha, \beta} \underbrace{\sum_{u,i} (\alpha + \beta_u + \beta_i - R_{u,i})^2}_{\text{error}} + \lambda \underbrace{[\sum_u \beta_u^2 + \sum_i \beta_i^2]}_{\text{regularizer}}$$

Jointly convex in β_i, β_u . Can be solved by iteratively removing the mean and solving for beta

Rating prediction

Iterative procedure – repeat the following updates until convergence:

$$\alpha = \frac{\sum_{u,i \in \text{train}} (R_{u,i} - (\beta_u + \beta_i))}{N_{\text{train}}}$$

$$\beta_u = \frac{\sum_{i \in I_u} R_{u,i} - (\alpha + \beta_i)}{\lambda + |I_u|}$$

$$\beta_i = \frac{\sum_{u \in U_i} R_{u,i} - (\alpha + \beta_u)}{\lambda + |U_i|}$$

(exercise: write down derivatives and convince yourself of these update equations!)

Rating prediction

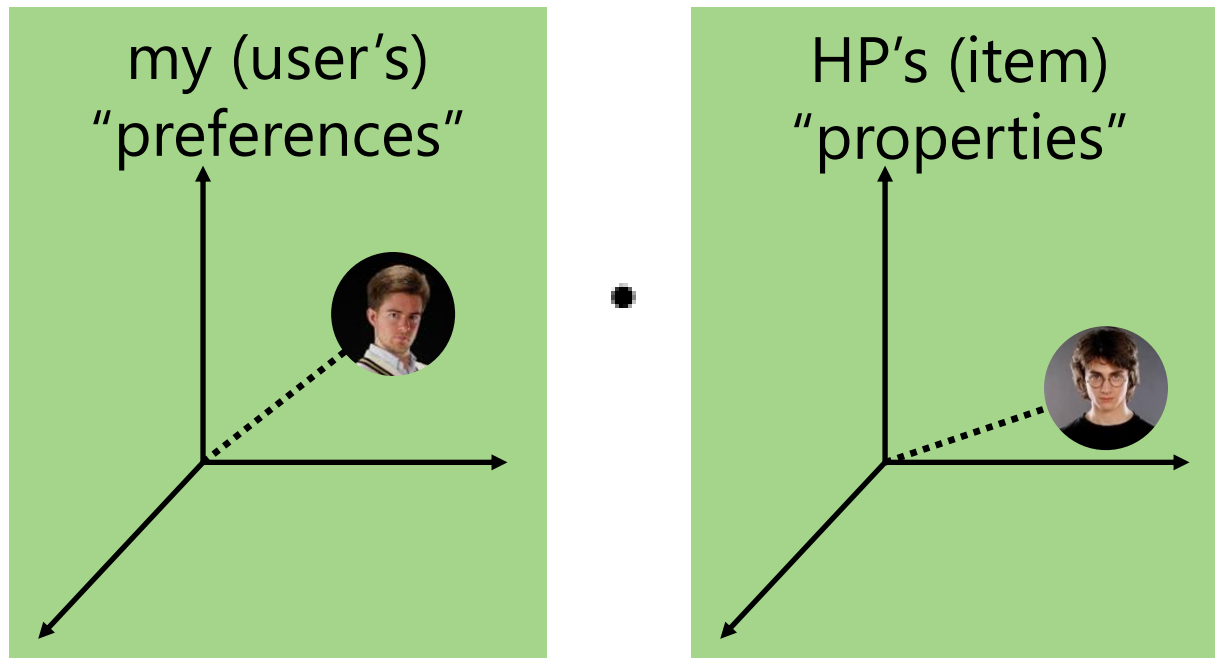
Looks good (and actually works surprisingly well), but doesn't solve the basic issue that we started with

$$\begin{aligned} f(\text{user features}, \text{movie features}) &= \\ &= \underbrace{\langle \phi(\text{user features}), \theta_{\text{user}} \rangle}_{\text{user predictor}} + \underbrace{\langle \phi(\text{movie features}), \theta_{\text{movie}} \rangle}_{\text{movie predictor}} \end{aligned}$$

That is, we're **still** fitting a function that treats users and items independently

Recommending things to people

How about an approach based on
dimensionality reduction?



i.e., let's come up with low-dimensional representations of the users and the items so as to best explain the data

Dimensionality reduction

We already have some tools that ought to help us, e.g. from lecture 3:

$$R = \begin{pmatrix} 5 & 3 & \cdots & 1 \\ 4 & 2 & & 1 \\ 3 & 1 & & 3 \\ 2 & 2 & & 4 \\ 1 & 5 & & 2 \\ \vdots & & \ddots & \vdots \\ 1 & 2 & \cdots & 1 \end{pmatrix}$$

What is the best low-rank approximation of R in terms of the mean-squared error?

Dimensionality reduction

We already have some tools that ought to help us, e.g. from lecture 3:

$$R = \begin{pmatrix} 5 & 3 & \cdots & 1 \\ 4 & 2 & & 1 \\ 3 & 1 & & 3 \\ \vdots & & \ddots & \vdots \\ 1 & 2 & \cdots & 1 \end{pmatrix}$$

Singular Value Decomposition

(square roots of)
eigenvalues of RR^T

$$R = U \Sigma V^T$$

eigenvectors of RR^T

eigenvectors of $R^T R$

The “best” rank-K approximation (in terms of the MSE) consists of taking the eigenvectors with the highest eigenvalues

Dimensionality reduction

But! Our matrix of ratings is only partially observed; and it's **really big!**

$$R = \begin{pmatrix} 5 & 3 & \dots & \cdot \\ 4 & 2 & & 1 \\ 3 & \cdot & & 3 \\ \cdot & 2 & & 4 \\ 1 & 5 & & \cdot \\ \vdots & & \ddots & \vdots \\ 1 & 2 & \dots & \cdot \end{pmatrix}$$

Missing ratings

SVD is **not defined** for partially observed matrices, and it is **not practical** for matrices with 1Mx1M+ dimensions

Latent-factor models

Instead, let's solve approximately using gradient descent

$$R = \begin{pmatrix} 5 & 3 & \cdots & \cdot \\ 4 & 2 & & 1 \\ 3 & \cdot & & 3 \\ \cdot & 2 & & 4 \\ 1 & 5 & & \cdot \\ \vdots & & \ddots & \vdots \\ 1 & 2 & \cdots & \cdot \end{pmatrix} \left. \vphantom{\begin{pmatrix} 5 \\ 4 \\ 3 \\ \cdot \\ 1 \\ \vdots \\ 1 \end{pmatrix}} \right\} \text{users}$$

$\underbrace{\hspace{10em}}_{\text{items}}$

K-dimensional representation of each item

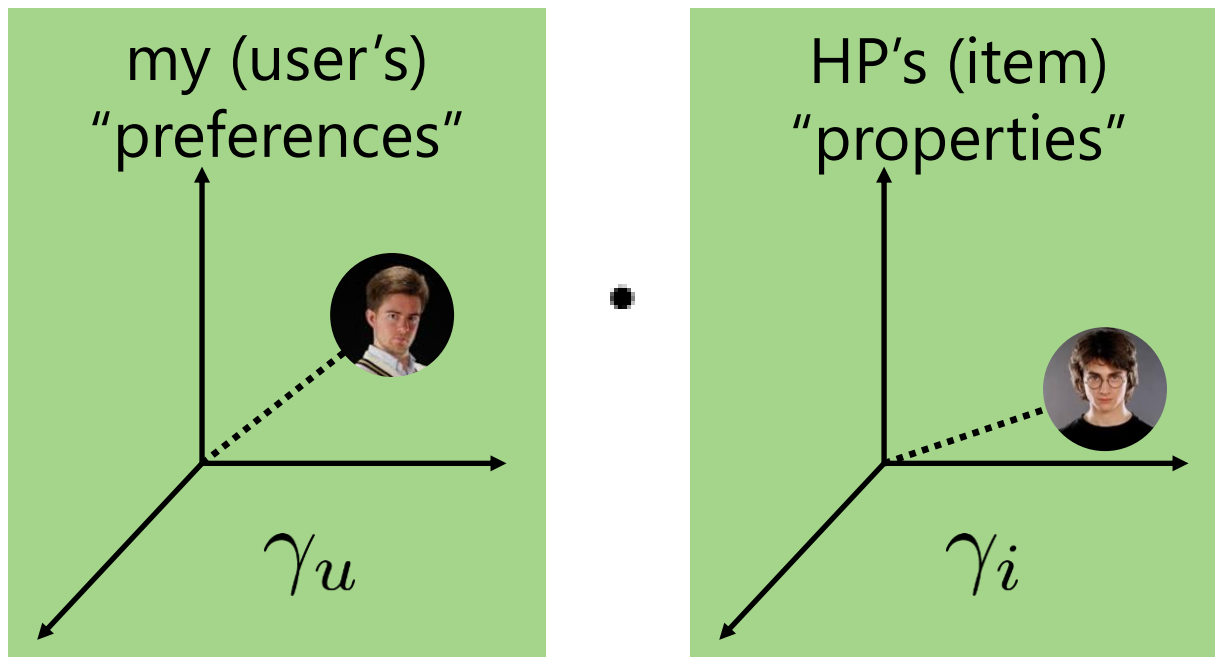
$$R \simeq UV^T$$

K-dimensional representation of each user

Latent-factor models

Let's write this as:

$$f(u, i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$



Latent-factor models

Let's write this as:

$$f(u, i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$

Our optimization problem is then

$$\arg \min_{\alpha, \beta, \gamma} \underbrace{\sum_{u,i} (\alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i - R_{u,i})^2}_{\text{error}} + \lambda \underbrace{[\sum_u \beta_u^2 + \sum_i \beta_i^2 + \sum_i \|\gamma_i\|_2^2 + \sum_u \|\gamma_u\|_2^2]}_{\text{regularizer}}$$

Problem: this is certainly not convex

(proof is easy: (1) it is smooth; (2) permuting the columns of gamma preserves the objective; (3) therefore it has multiple local optima and cannot be convex;

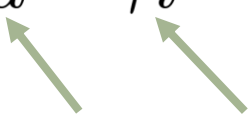
(4) in other words it must look like this: 

permutations of local minima

Latent-factor models

Oh well. We'll just solve it approximately

Observation: if we know either the user or the item parameters, the problem becomes easy

$$f(u, i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$


e.g. fix γ_i – pretend we're fitting parameters for features

Latent-factor models

This gives rise to a simple (though approximate) solution

objective:

$$\arg \min_{\alpha, \beta, \gamma} \sum_{u,i} (\alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i - R_{u,i})^2 + \lambda [\sum_u \beta_u^2 + \sum_i \beta_i^2 + \sum_i \|\gamma_i\|_2^2 + \sum_u \|\gamma_u\|_2^2]$$

$$= \arg \min_{\alpha, \beta, \gamma} \text{objective}(\alpha, \beta, \gamma)$$

1) fix γ_i . Solve $\arg \min_{\alpha, \beta, \gamma_u} \text{objective}(\alpha, \beta, \gamma)$

2) fix γ_u . Solve $\arg \min_{\alpha, \beta, \gamma_i} \text{objective}(\alpha, \beta, \gamma)$

3,4,5...) repeat until convergence

Each of these subproblems is “easy” – just regularized least-squares, like we’ve been doing since week 1. This procedure is called **alternating least squares**.

Latent-factor models

Observation: we went from a method which uses **only** features:

$f(\text{user features, movie features}) \rightarrow \text{star rating}$

User features:
age, gender,
location, etc.

A. Phillips
Reviewer ranking: #17,230,554

90% helpful
votes received on reviews
(151 of 167)

ABOUT ME
Enjoy the reviews...

ACTIVITIES
Reviews (16)

Movie features: genre,
actors, rating, length, etc.

Product Details

Genres	Science Fiction, Action, Horror
Director	David Twohy
Starring	Vin Diesel, Radha Mitchell
Supporting actors	Cole Hauser, Keith David, Lewis Fitz-Gerald, Claudia Black, Rhiana Granger, Angela Moore, Peter Chiang, Ken Twohy
Studio	NBC Universal
MPAA rating	R (Restricted)
Captions and subtitles	English Details
Rental rights	24 hour viewing period. Details
Purchase rights	Stream instantly and download to 2 locations Details
Format	Amazon Instant Video (streaming online video and digital download)

to one which **completely ignores** them:

$$\arg \min_{\alpha, \beta, \gamma} \sum_{u,i} (\alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i - R_{u,i})^2 + \lambda [\sum_u \beta_u^2 + \sum_i \beta_i^2 + \sum_i \|\gamma_i\|_2^2 + \sum_u \|\gamma_u\|_2^2]$$

Latent-factor models

Should we use features or not?

1) Argument **against** features:

Imagine incorporating features into the model like:

$$f(u, i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i + \langle \phi(u), \theta_u \rangle + \langle \phi(i), \theta_i \rangle$$

which is equivalent to:

$$f(u, i) = \alpha + \beta_u + \beta_i + \underbrace{(\phi(u); \phi(i); \gamma_u)}_{\text{knowns}} \cdot \underbrace{(\theta_u; \theta_i; \gamma_i)}_{\text{unknowns}}$$

but this has fewer degrees of freedom than a model which replaces the knowns by unknowns:

$$f(u, i) = \alpha + \beta_u + \beta_i + (\gamma'_i; \gamma'_u; \gamma_u) \cdot (\theta_u; \theta_i; \gamma_i)$$

Latent-factor models

Should we use features or not?

1) Argument **against** features:

So, the addition of features adds **no expressive power** to the model. We **could** have a feature like “is this an action movie?”, but if this feature were useful, the model would “discover” a latent dimension corresponding to action movies, and we wouldn’t need the feature anyway

In the limit, this argument is valid: as we add more ratings per user, and more ratings per item, the latent-factor model should automatically discover any useful dimensions of variation, so the influence of observed features will disappear

Latent-factor models

Should we use features or not?

2) Argument **for** features:

But! Sometimes we **don't** have many ratings per user/item

Latent-factor models are next-to-useless if **either** the user or the item was never observed before

$$f(u, i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$

reverts to zero if we've never seen the user before
(because of the regularizer)

Latent-factor models

Should we use features or not?

2) Argument **for** features:

This is known as the **cold-start** problem in recommender systems. Features are not useful if we have many observations about users/items, but are useful for **new** users and items.

We also need some way to handle users who are **active**, but don't necessarily rate anything, e.g. through **implicit feedback**

Overview & recap

Tonight we've followed the programme below:

1. Measuring similarity between users/items for **binary** prediction (e.g. Jaccard similarity)
2. Measuring similarity between users/items for **real-valued** prediction (e.g. cosine/Pearson similarity)
3. Dimensionality reduction for **real-valued** prediction (latent-factor models)
4. **Finally** – dimensionality reduction for **binary** prediction

One-class recommendation

How can we use **dimensionality reduction** to predict **binary** outcomes?

- In weeks 1&2 we saw **regression** and **logistic regression**. These two approaches use the same type of linear function to predict real-valued and binary outputs
- We can apply an analogous approach to binary recommendation tasks

One-class recommendation

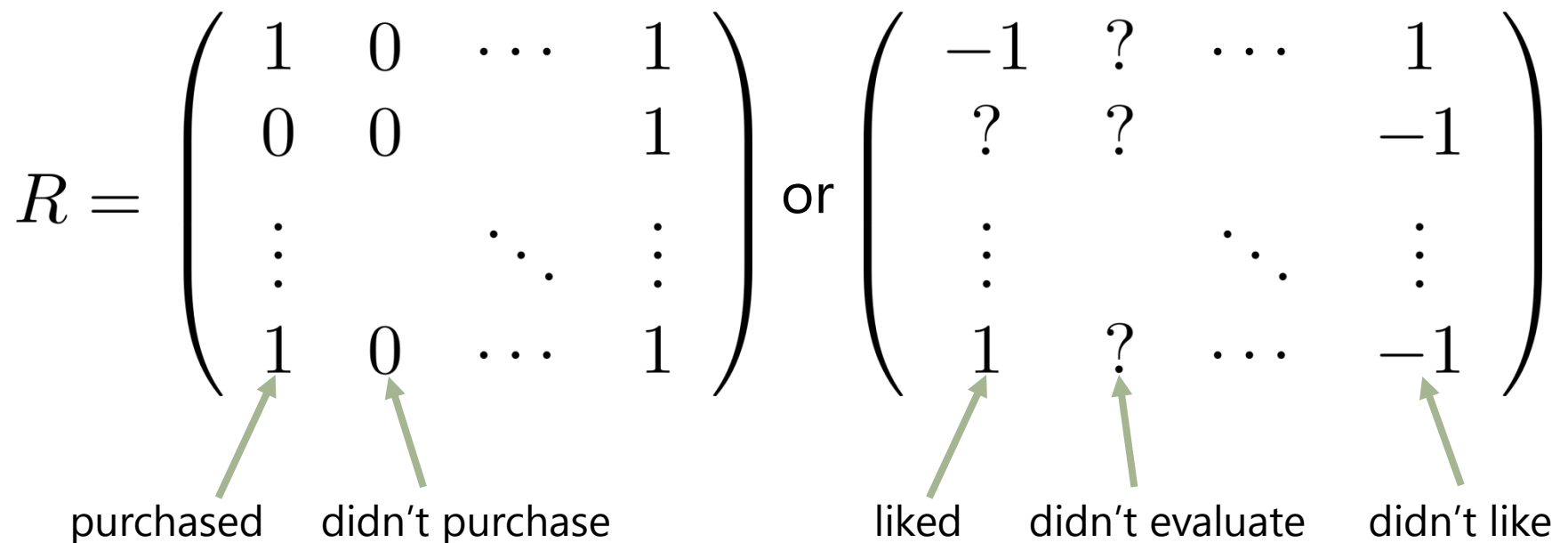
This is referred to as “**one-class**” recommendation

- In weeks 1&2 we saw **regression** and **logistic regression**. These two approaches use the same type of linear function to predict real-valued and binary outputs
- We can apply an analogous approach to binary recommendation tasks

One-class recommendation

Suppose we have binary (0/1) observations
(e.g. purchases) or positive/negative
feedback (thumbs-up/down)

$$R = \begin{pmatrix} 1 & 0 & \cdots & 1 \\ 0 & 0 & & 1 \\ \vdots & & \ddots & \vdots \\ 1 & 0 & \cdots & 1 \end{pmatrix} \text{ or } \begin{pmatrix} -1 & ? & \cdots & 1 \\ ? & ? & & -1 \\ \vdots & & \ddots & \vdots \\ 1 & ? & \cdots & -1 \end{pmatrix}$$



purchased didn't purchase liked didn't evaluate didn't like

One-class recommendation

So far, we've been fitting functions of the form

$$R \simeq UV^T$$

- Let's change this so that we maximize the **difference** in predictions between positive and negative items
- E.g. for a user who likes an item i and dislikes an item j we want to maximize:

$$\max \ln \sigma(\gamma_u \cdot \gamma_i - \gamma_u \cdot \gamma_j)$$

One-class recommendation

We can think of this as maximizing the probability of correctly predicting pairwise preferences, i.e.,

$$p(i \text{ is preferred over } j) = \sigma(\gamma_u \cdot \gamma_i - \gamma_u \cdot \gamma_j)$$

- As with logistic regression, we can now maximize the likelihood associated with such a model by gradient ascent
 - In practice it isn't feasible to consider all pairs of positive/negative items, so we proceed by stochastic gradient ascent – i.e., randomly sample a (positive, negative) pair and update the model according to the gradient w.r.t. that pair

Summary

Recap

1. Measuring similarity between users/items for **binary** prediction
Jaccard similarity
2. Measuring similarity between users/items for **real-valued** prediction
cosine/Pearson similarity
3. Dimensionality reduction for **real-valued** prediction
latent-factor models
4. Dimensionality reduction for **binary** prediction
one-class recommender systems

Questions?

Further reading:

One-class recommendation:

<http://goo.gl/08Rh59>

Amazon's solution to collaborative filtering at scale:

<http://www.cs.umd.edu/~samir/498/Amazon-Recommendations.pdf>

An (expensive) textbook about recommender systems:

<http://www.springer.com/computer/ai/book/978-0-387-85819-7>

Cold-start recommendation (e.g.):

<http://wanlab.poly.edu/recsys12/recsys/p115.pdf>

CSE 255 – Lecture 5

Data Mining and Predictive Analytics

Extensions of latent-factor models,
(and more on the Netflix prize!)

Extensions of latent-factor models

So far we have a model that looks like:

$$f(u, i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$

How might we extend this to:

- Incorporate features about users and items
 - Handle implicit feedback
 - Change over time

See **Yehuda Koren** (+Bell & Volinsky)'s magazine article:
"Matrix Factorization Techniques for Recommender Systems"
IEEE Computer, 2009

Extensions of latent-factor models

1) Features about users and/or items

(simplest case) Suppose we have **binary attributes** to describe users or items

$$A(u) = [1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1]$$

attribute vector for user u

e.g. is female

is male

is between 18-24yo

Extensions of latent-factor models

1) Features about users and/or items

(simplest case) Suppose we have **binary attributes** to describe users or items

- Associate a **parameter vector** with each attribute
- Each vector encodes how much a particular feature "offsets" the given latent dimensions

$$A(u) = [1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1]$$

attribute vector for user u

e.g. $y_0 = [-0.2, 0.3, 0.1, -0.4, 0.8]$
~ "how does being male impact γ_u "

Extensions of latent-factor models

1) Features about users and/or items

(simplest case) Suppose we have **binary attributes** to describe users or items

- Associate a **parameter vector** with each attribute
- Each vector encodes how much a particular feature "offsets" the given latent dimensions
 - Model looks like:

$$f(u, i) = \alpha + \beta_u + \beta_i + \left(\gamma_u + \sum_{a \in A(u)} \rho_a \right) \cdot \gamma_i$$

- Fit as usual:

$$\arg \min_{\alpha, \beta, \gamma, \rho} \underbrace{\sum_{u, i \in \text{train}} (f(u, i) - r_{u, i})^2}_{\text{error}} + \underbrace{\lambda \Omega(\beta, \gamma)}_{\text{regularizer}}$$

Extensions of latent-factor models

2) Implicit feedback

Perhaps many users will never actually rate things, but may still interact with the system, e.g. through the movies they view, or the products they purchase (but never rate)

- Adopt a similar approach – introduce a binary vector describing a user's actions

$$N(u) = [1, 0, 0, 0, 1, 0, \dots, 0, 1]$$

implicit feedback vector for user u

e.g. $y_{u0} = [-0.1, 0.2, 0.3, -0.1, 0.5]$
Clicked on "Love Actually" but didn't watch

Extensions of latent-factor models

2) Implicit feedback

Perhaps many users will never actually rate things, but may still interact with the system, e.g. through the movies they view, or the products they purchase (but never rate)

- Adopt a similar approach – introduce a binary vector describing a user's actions
 - Model looks like:

$$f(u, i) = \alpha + \beta_u + \beta_i + \left(\gamma_u + \frac{1}{\|N(u)\|} \sum_{a \in N(u)} \rho_a \right) \cdot \gamma_i$$


normalize by the number of actions the user performed

Extensions of latent-factor models

3) Change over time

There are a number of reasons why rating data might be subject to temporal effects...

Extensions of latent-factor models

3) Change over time

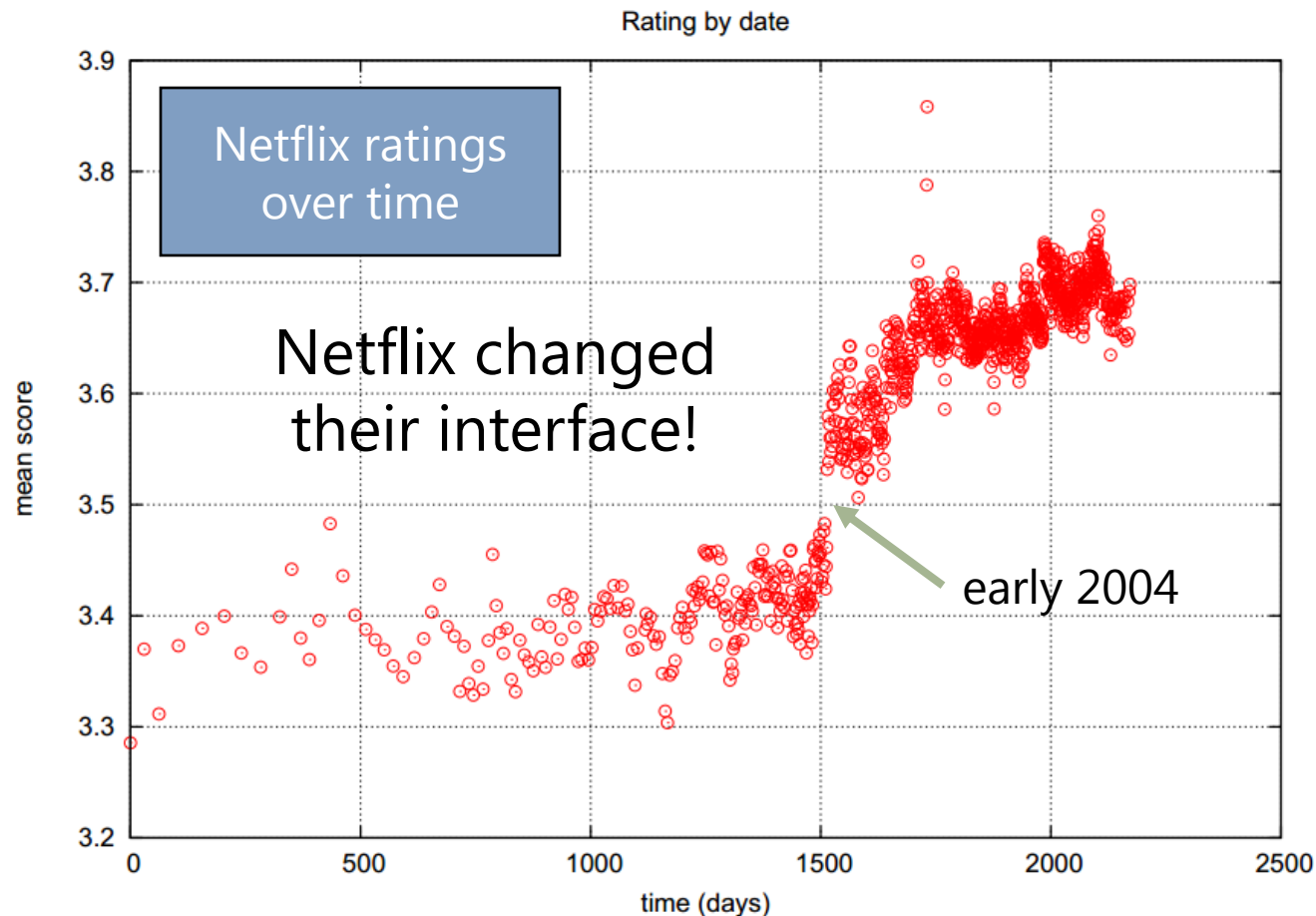


Figure from Koren: "Collaborative Filtering with Temporal Dynamics" (KDD 2009)

Extensions of latent-factor models

3) Change over time

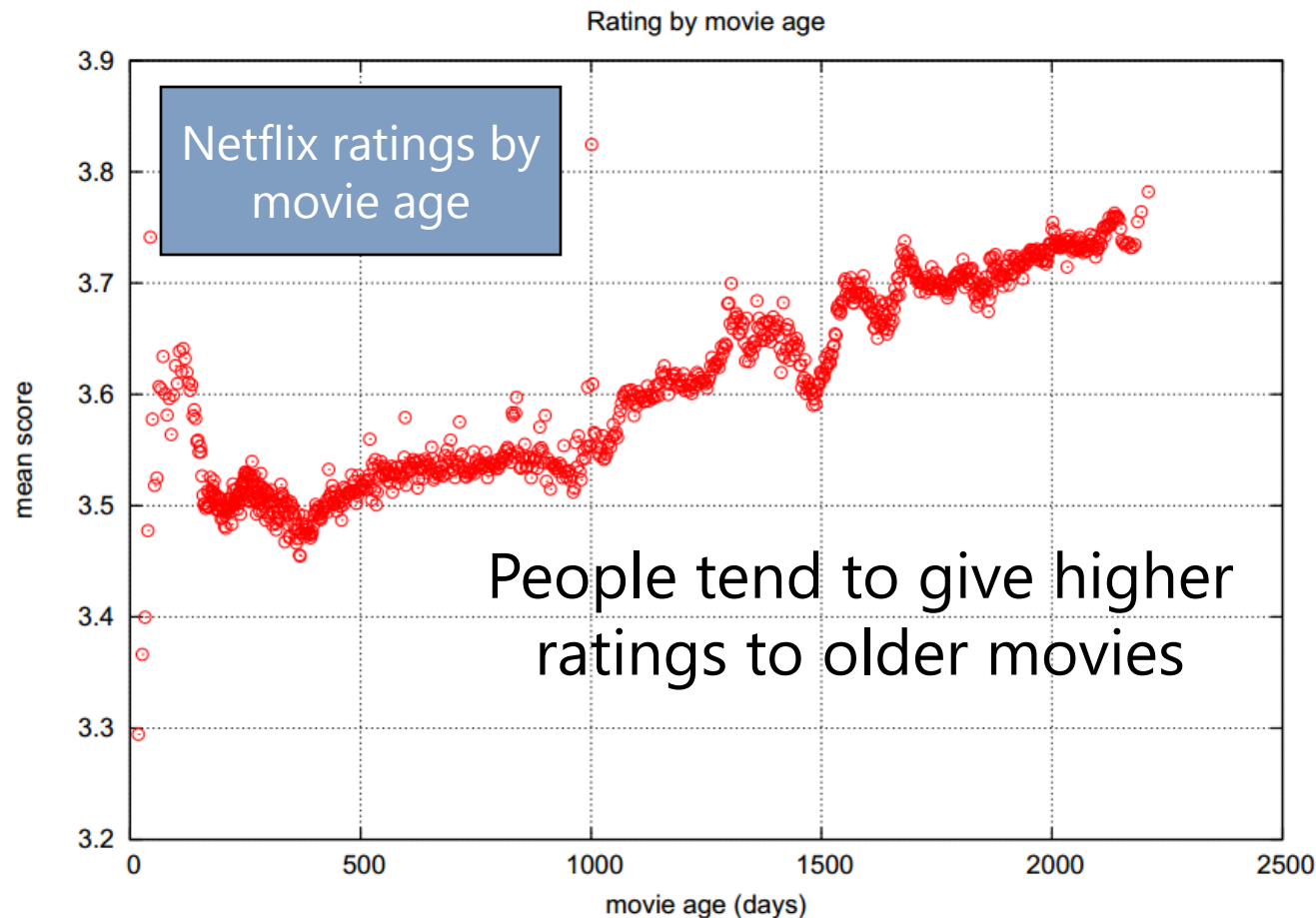
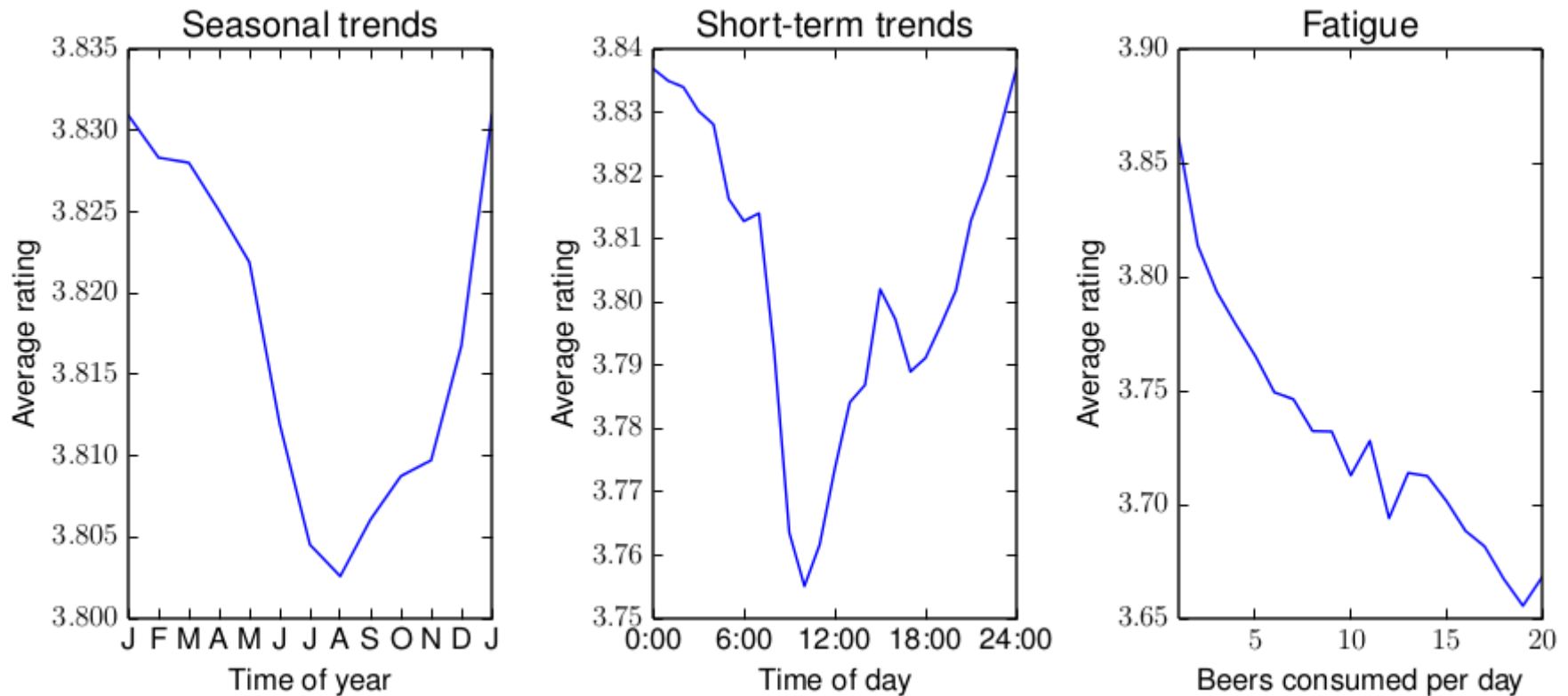


Figure from Koren: "Collaborative Filtering with Temporal Dynamics" (KDD 2009)

Extensions of latent-factor models

3) Change over time

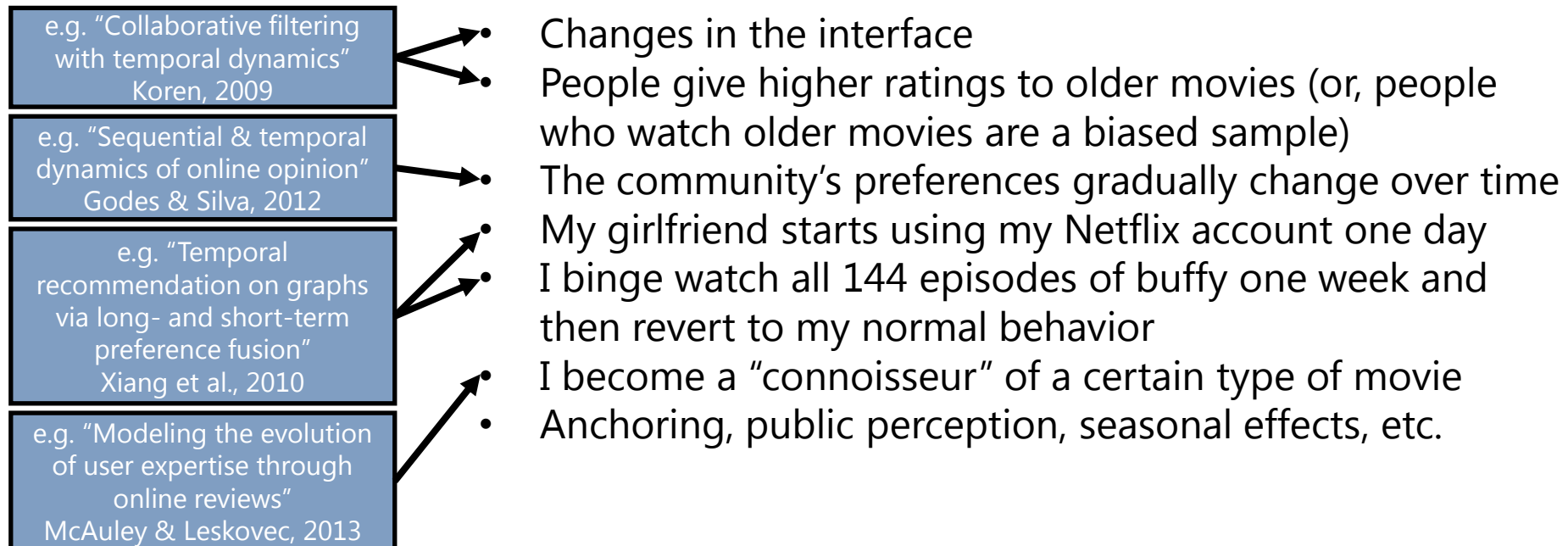


A few temporal effects from beer reviews

Extensions of latent-factor models

3) Change over time

There are a number of reasons why rating data might be subject to temporal effects...



Extensions of latent-factor models

3) Change over time

Each definition of temporal evolution demands a slightly different model assumption (we'll see some in more detail later tonight!) but the basic idea is the following:

1) Start with our original model:

$$f(u, i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$

2) And define some of the parameters as a function of time:

$$f(u, i, t) = \alpha + \beta_u(t) + \beta_i(t) + \gamma_u(t) \cdot \gamma_i$$

3) Add a regularizer to constrain the time-varying terms:

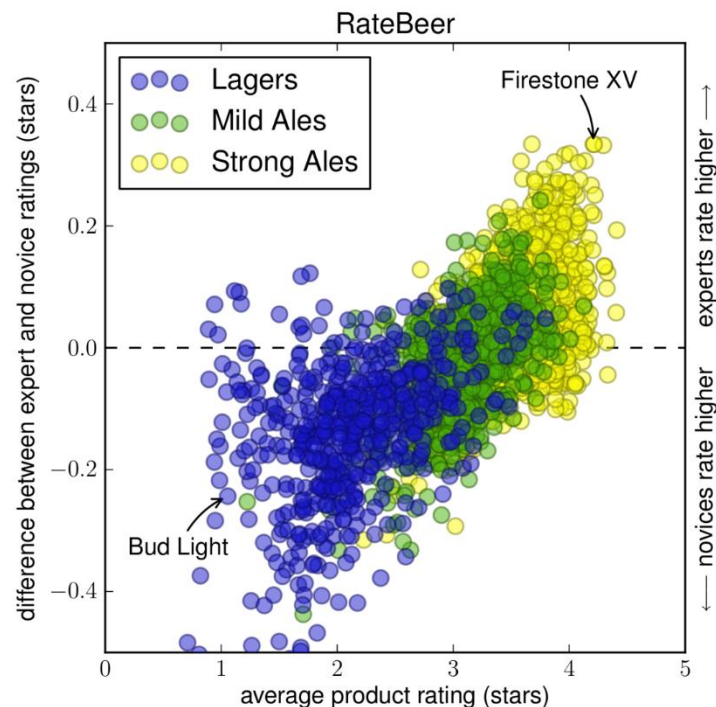
$$\arg \min_{\alpha, \beta, \gamma} \sum_{u, i, t \in \text{train}} (f(u, i, t) - r_{u, i, t})^2 + \lambda_1 \Omega(\beta, \gamma) + \underbrace{\lambda_2 \|\gamma(t) - \gamma(t + \delta)\|}_{\text{parameters should change smoothly (I'll give an example in the set of slides after the break)}}$$

parameters should change smoothly (I'll give an example in the set of slides after the break)

Extensions of latent-factor models

3) Change over time

After the break: how do people acquire tastes for beers (and potentially for other things) over time?



Differences between
"beginner" and "expert"
preferences for different
beer styles

Extensions of latent-factor models

4) Missing-not-at-random

- Our decision about whether to purchase a movie (or item etc.) is a function of how we **expect** to rate it
- Even for items we've purchased, our decision to **enter a rating** or write a review **is a function of our rating**
 - e.g. some rating distribution from a few datasets:

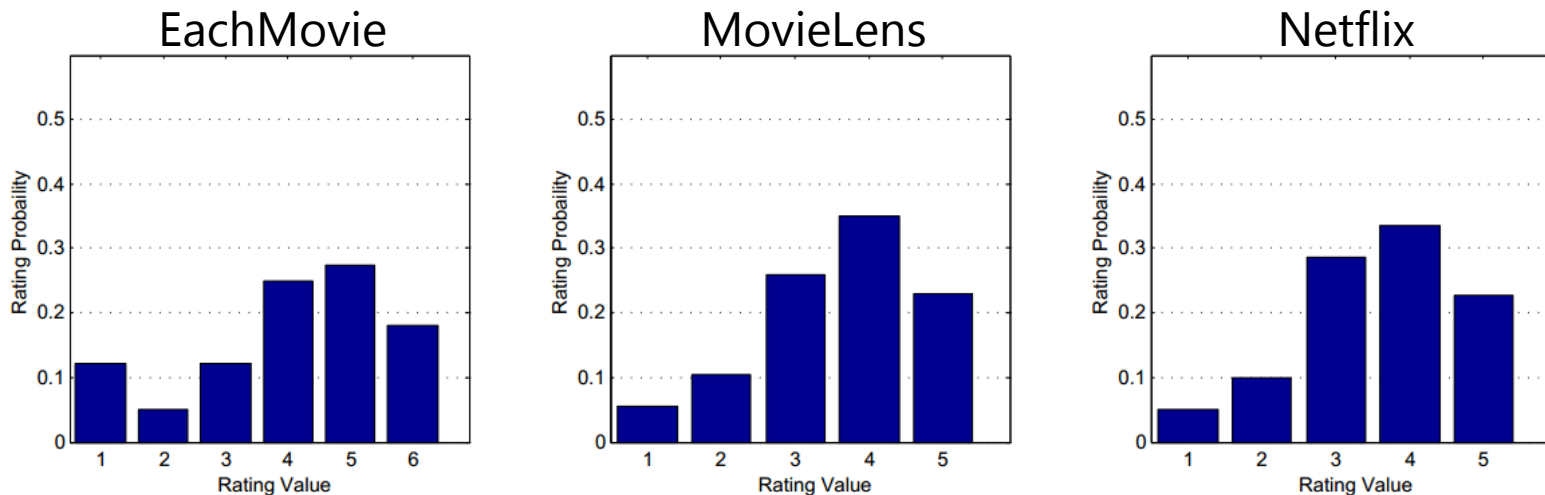
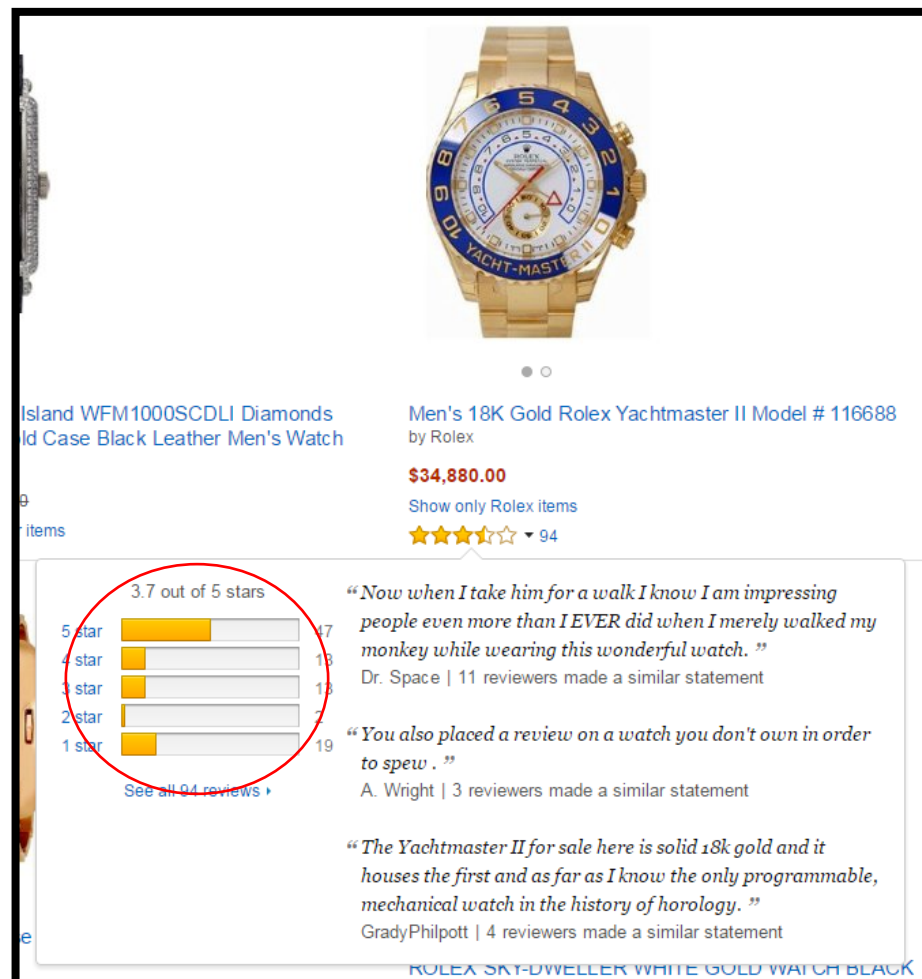


Figure from Marlin et al. "Collaborative Filtering and the Missing at Random Assumption" (UAI 2007)

Extensions of latent-factor models

4) Missing-not-at-random

e.g. Men's watches:



Island WFM1000SCDLI Diamonds
Old Case Black Leather Men's Watch

Men's 18K Gold Rolex Yachtmaster II Model # 116688
by Rolex

\$34,880.00

Show only Rolex items

★★★★☆ 94

3.7 out of 5 stars

Star Rating	Count
5 star	47
4 star	18
3 star	18
2 star	2
1 star	19

See all 94 reviews ▶

"Now when I take him for a walk I know I am impressing people even more than I EVER did when I merely walked my monkey while wearing this wonderful watch. "
Dr. Space | 11 reviewers made a similar statement

"You also placed a review on a watch you don't own in order to spew . "
A. Wright | 3 reviewers made a similar statement

"The Yachtmaster II for sale here is solid 18k gold and it houses the first and as far as I know the only programmable, mechanical watch in the history of horology. "
GradyPhilpott | 4 reviewers made a similar statement

ROLEX SKY-DWELLER WHITE GOLD WATCH BLACK

Extensions of latent-factor models

4) Missing-not-at-random

- Our decision about whether to purchase a movie (or item etc.) is a function of how we **expect** to rate it
- Even for items we've purchased, our decision to **enter a rating** or write a review **is a function of our rating**
 - So we can predict ratings more accurately by building models that account for these differences
 1. Not-purchased items have a different prior on ratings than purchased ones
 2. Purchased-but-not-rated items have a different prior on ratings than rated ones

Moral(s) of the story

How much do these extensions help?

Moral: increasing complexity helps a bit, but changing the model can help **a lot**

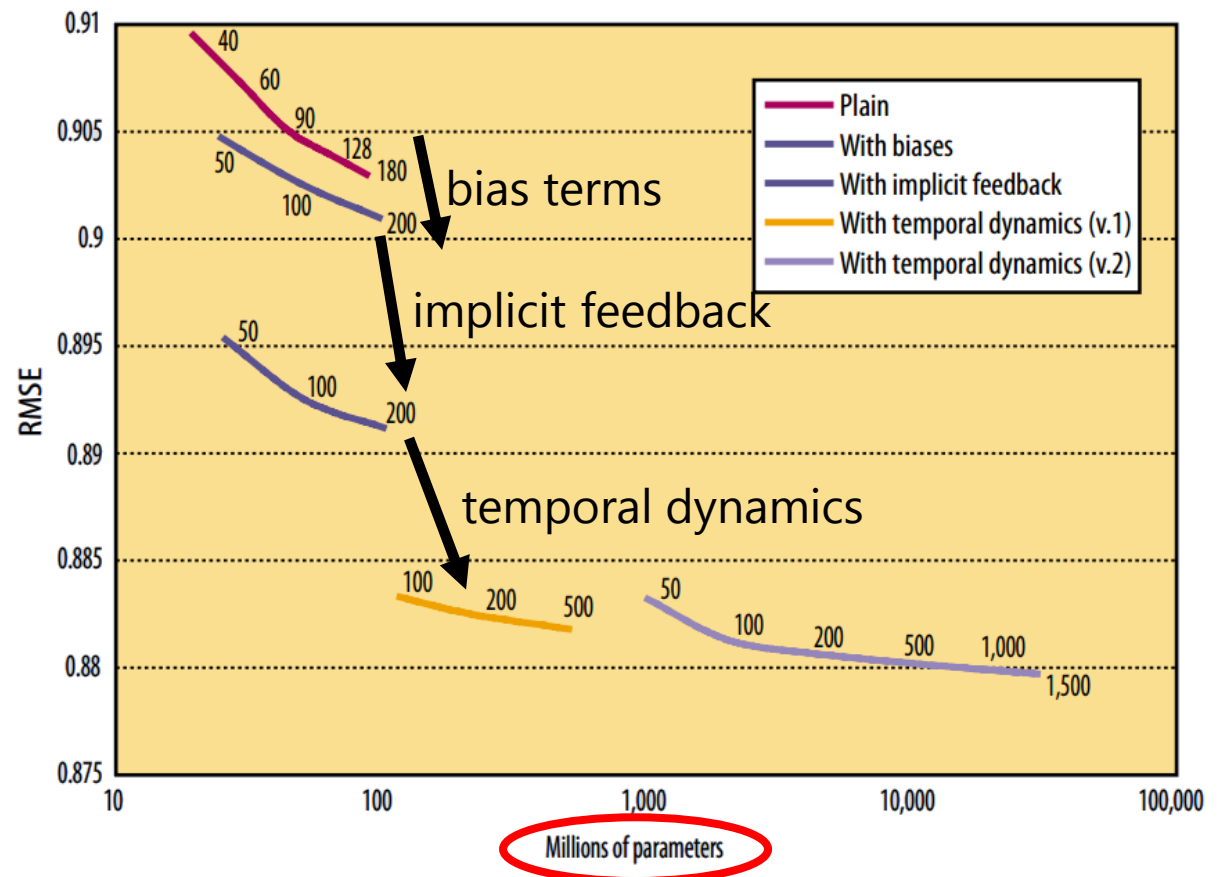


Figure from Koren: "Collaborative Filtering with Temporal Dynamics" (KDD 2009)

Moral(s) of the story

So what actually happened with Netflix?

- The AT&T team “BellKor”, consisting of Yehuda Koren, Robert Bell, and Chris Volinsky were early leaders. Their main insight was how to effectively incorporate temporal dynamics into recommendation on Netflix.
- Before long, it was clear that no one team would build the winning solution, and Frankenstein efforts started to merge. Two frontrunners emerged, “BellKor’s Pragmatic Chaos”, and “The Ensemble”.
- The BellKor team was the first to achieve a 10% improvement in RMSE, putting the competition in “last call” mode. The winner would be decided after 30 days.
- After 30 days, performance was evaluated on the hidden part of the test set.
- Both of the frontrunning teams had **the same** RMSE (up to some precision) but BellKor’s team submitted their solution 20 minutes earlier and won \$1,000,000

For a less rough summary, see the Wikipedia page about the Netflix prize, and the nytimes article about the competition: <http://goo.gl/WNpy7o>

Moral(s) of the story

Afterword

- Netflix had a class-action lawsuit filed against them after somebody de-anonymized the competition data
- \$1,000,000 seems to be **incredibly cheap** for a company the size of Netflix in terms of the amount of research that was devoted to the task, and the potential benefit to Netflix of having their recommendation algorithm improved by 10%
- Other similar competitions have emerged, such as the Heritage Health Prize (\$3,000,000 to predict the length of future hospital visits)
- But... the winning solution never made it into production at Netflix – it's a monolithic algorithm that is very expensive to update as new data comes in*

*source: a friend of mine told me and I have no actual evidence of this claim

Moral(s) of the story

Finally...

Q: Is the RMSE really the right approach? Will improving rating prediction by 10% actually improve the user experience by a significant amount?

A: Not clear. Even a solution that only changes the RMSE slightly could drastically change which items are top-ranked and ultimately suggested to the user.

Q: But... are the following recommendations actually any good?

A1: Yes, these are my favorite movies!

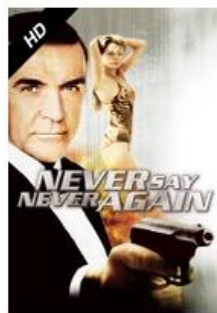
or **A2:** No! There's no **diversity**, so how will I discover **new** content?



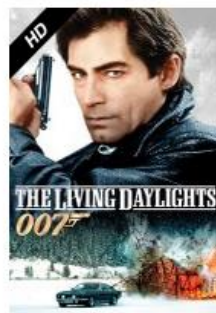
5.0 stars



5.0 stars



5.0 stars



5.0 stars



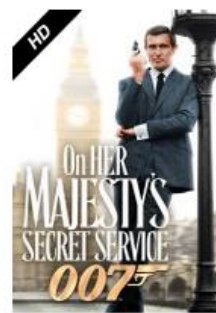
4.9 stars



4.9 stars



4.8 stars



4.8 stars

predicted rating

Summary

Various extensions of latent factor models:

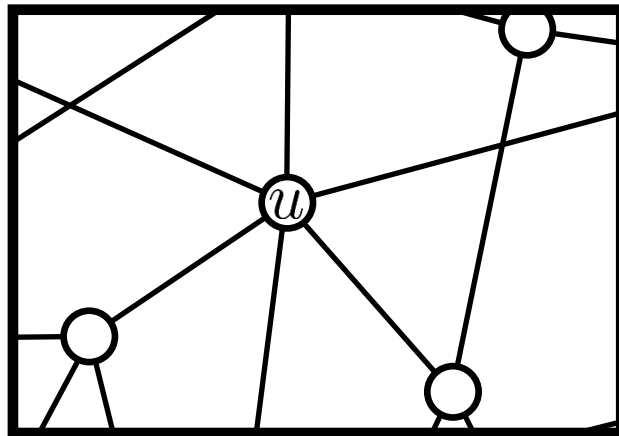
- Incorporating features
e.g. for cold-start recommendation
 - Implicit feedback
e.g. when ratings aren't available, but other actions are
- Incorporating temporal information into latent factor models
seasonal effects, short-term "bursts", long-term trends, etc.
 - Missing-not-at-random
incorporating priors about items that were not bought or rated
 - The Netflix prize

Things I didn't get to...

Socially regularized recommender systems

see e.g. "Recommender Systems with Social Regularization"

<http://research.microsoft.com/en-us/um/people/denzho/papers/rsr.pdf>



network

$$\arg \min_{\alpha, \beta, \gamma} \sum_{u, i \in \text{train}} (f(u, i) - r_{u, i})^2 + \lambda_1 \Omega(\beta, \gamma) + \underbrace{\lambda_2 \sum_{u, v \in \mathcal{E}} \|\gamma_u - \gamma_v\|}_{\text{social regularizer}}$$

Things I didn't get to...

Online advertising

Recommendation in certain settings (e.g. online advertising) has drastically different assumptions compared to what's appropriate for products on Amazon or movies on Netflix. e.g.

- I want to show the ad I believe you are most likely to click on
- But, I also want to discover your preferences for categories of ads about which I have no information
- So there is a natural **exploration/exploitation** tradeoff when making recommendations

See e.g. "A Contextual-Bandit Approach to Personalized News Article Recommendation"

<http://www.research.rutgers.edu/~lihong/pub/Li10Contextual.pdf>

Questions?

Further reading:

Yehuda Koren's, Robert Bell, and Chris Volinsky's IEEE computer article:

<http://www2.research.att.com/~volinsky/papers/ieeecomputer.pdf>

Paper about the "Missing-at-Random" assumption, and how to address it:

<http://www.cs.toronto.edu/~marlin/research/papers/cfmar-uai2007.pdf>

Collaborative filtering with temporal dynamics:

<http://research.yahoo.com/files/kdd-fp074-koren.pdf>

Recommender systems and sales diversity:

http://papers.ssrn.com/sol3/papers.cfm?abstract_id=955984

Up next: Assignment 2!